Bauhaus-Universität Weimar

SPOT as a companion in AEC industry project sites

Abhishek Dilip Patil,
Matriculation Nr. – 127126
Niraj Kishore Ladhe
Matriculation Nr. – 125815

Contents

List of figures

List of Tables

Abstract

The Architecture, Engineering and Construction (AEC) Industry increasingly demands accurate, frequent, and automated monitoring of project progress and quality to reduce costs, improve safety, and ensure alignment with design and regulatory standards. The traditional methods are resource intensive, time – consuming, and often expose personnel to hazardous environments; moreover, multisector stakeholders introduce discrepancies in execution that are detected too late.

This project investigated the integration of a quadruped robot – Boston Dynamics' SPOT – into construction site monitoring. SPOT was coded and tested to perform two key functions: autonomous navigation supported by fiducial tag localization, with manual override capability to address complex site conditions. SPOT can capture site images at predefined locations or perform tasks such as movement or digitally communicating with a client system, with the potential of interacting with the environment, although this aspect was beyond the scope of the project.

Based on the results, the study demonstrates SPOT's potential effectiveness as a companion for construction professionals to perform labour-intensive monitoring tasks and demonstrates its potential to enhance safety and efficiency on site.

Keywords –

SPOT, quadruped robot, Client system, server, API, fiducial tags, RPC, gRPC, protobuf.

1 Introduction

1.1 Context and Motivation

Construction projects are inherently complex and prone to uncertainty, making continuous monitoring and performance control essential for successful delivery. Traditional inspection and monitoring methods remain largely manual, resulting in cost overruns, delays, and reduced efficiency. They also fail to provide timely, accurate information, limiting project managers' ability to respond to deviations effectively [1].

Furthermore, a significant proportion of inspection time is spent on visual data collection and descriptive reporting, rather than on predictive or evaluative tasks that add greater value to project performance [1]. This inefficiency stems from the lack of automated, real-time monitoring systems that can provide reliable and accessible data to all stakeholders.

Many robotic monitoring systems remain at the proof-of-concept stage, with limited demonstrations in real-world construction environments [2]. The quadruped robots stand out with more effectiveness in traversing complex terrain which is usually found in construction sites including stairs. Existing implementations are typically fully autonomous solutions with no overriding manual control [3]. In such cases, the entire program needs to be switched to a manual control program, preventing seamless transitions between autonomy and human control. By leveraging their mobility and ability to operate in hazardous conditions, quadruped robots have the potential to transform site monitoring into a safer, more efficient, and proactive process. This creates a strong motivation to explore how quadruped robots can be systematically integrated into construction workflows. Not only as tools for data capture but as active companions for construction professionals with a dual autonomous-manual control [4].

1.2 Problem Statement

Despite advances in automation, construction monitoring systems are still constrained by the lack of flexible control modes. Existing implementations of quadruped robots are typically either fully autonomous or fully manual, requiring complete switching between modes rather than allowing real-time human intervention. This limitation reduces adaptability in dynamic and unpredictable construction environments, where both autonomy and manual control are necessary [2–5].

The specific gap lies in the absence of a system that enables SPOT to perform autonomous tasks like fiducial-based navigation, undertaking some tasks if a particular fiducial is detected while simultaneously having manual control when needed. Such dual functionality is essential for integrating quadruped robots effectively into construction workflows.

1.3 Scope

1. Understand the API of Boston Dynamics SPOT and create a program which can detect fiducial tags in the environment.
2. Assign each fiducial tag to a certain task or digital signal to the client system.
3. Have manual control over SPOT even in fiducial follow mode.
4. Development of a basic interface for the program.
5. Test the developed program and evaluate the results.

1.4 Limitations

1. The program could not be tested at an actual construction site due to lack of access; testing was limited to controlled environments.

2 Literature Review

At present a lot of sensors and digital equipment are used in the AEC industry. Out of them few implement certain level of robotic application. But very few use highly advance robotic technology as a quadruped robot.

A study shows a methodology to capture the site reality with a 360° camera and convert it in Unity to have a virtual environment. The camera is mounted on the quadruped robot. The captured images are then used to visualize the images in the virtual environment as discrete points of reality. This study is useful for construction mangers and inspector to potentially reduce manpower in inspections of construction projects [4,5]. Although the study does not mention of having manual control over SPOT while operation. Hence the need to have a dual functionality program.

In other case study SPOT is used by the Hamburg Port authority (HPA) for performing structural inspections in the cavities of Köhlbrand Bridge. The Köhlbrand Bridge, which is about 3.8 km long. The second longest road bridge in Germany, it has been the most important east-west link in the Port of Hamburg for over 50 years. Around 38,000 vehicles cross the bridge every day. This inspection takes months to complete and usually involves working in difficult conditions. The environment is dusky, dark, noisy, hot in summer and cold in winter. Even communication is difficult for distance more than 10m due to strong echoes. This was the perfect job for SPOT to undertake with predefined paths and mounting

various sensors like a laser scanner to create a digital twin of the internal cavities and then using various software to detect cracks in the structures [3]. The SPOT is usually given a predefined path on and then more focus is given on the points where cracks or damage is observed. Again, lacking dual functionality as well as SPOT does not have complete freedom to deviate from the designated path. The project aims to create a program which does not have a designated path and relies either on fiducial tags for path creation or manual control.

These studies demonstrate the potential of quadruped robots in construction monitoring; however, none integrate autonomous navigation with real-time manual override. This gap is the focus of the present project.

3 Methodology

Github is used for version control and code workflow [6].

3.1 System Architecture

High-level architecture of the SPOT monitoring system. The operator can issue **manual commands at any time** (Manual Override Module) while fiducial follow executes fiducial-triggered tasks. Perception feeds fiducial detection; the Task Manager performs actions (e.g., image capture, movement, digital signal to the client). Safety (E-Stop, lease, auth) supervises all actuation. Telemetry, images, and signals flow to the client system and optional storage/BIM server.

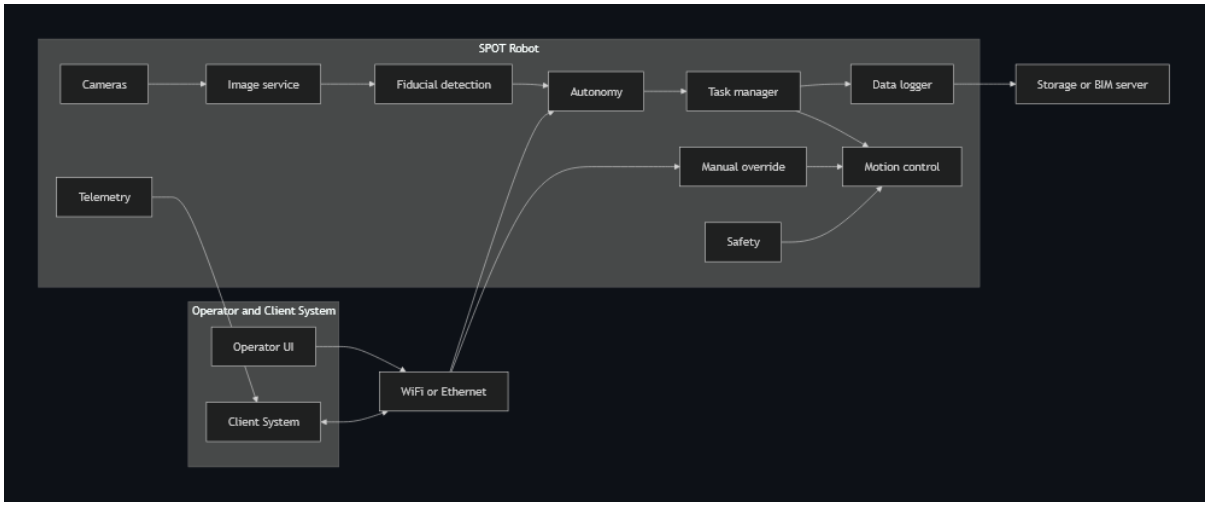

Figure 1:High level System Architecture of the spot program

3.1.1 Hardware

1. About SPOT

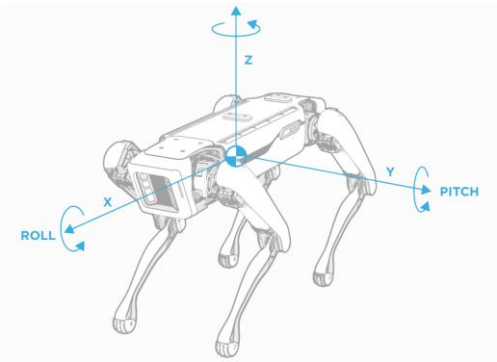

Figure 2: SPOT axes that are used for motion definition

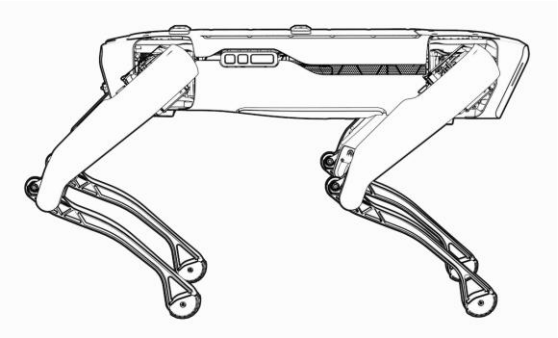

Figure 4: Side view of SPOT

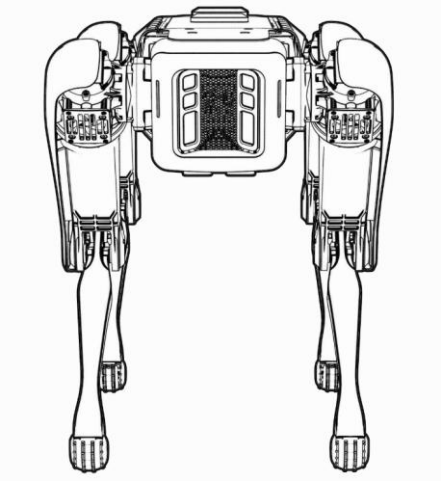

Figure 3: Front view of SPOT

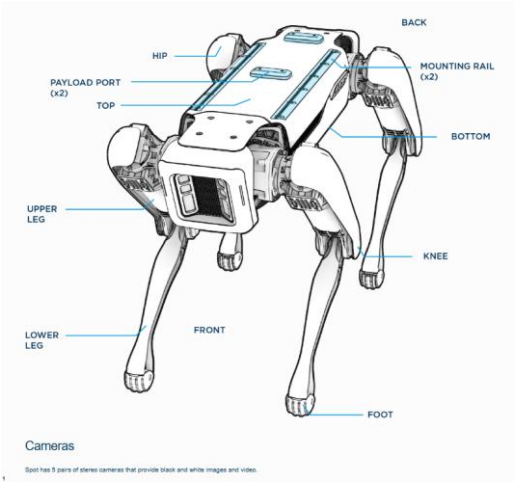

Figure 5: Basic labelled diagram of SPOT

Specification	Value
Length	1100 mm
Width	500 mm
Height (standing)	840 mm
Height (Seating)	191 mm
Weight	32.5 kg
DOF	12
Speed	1.6 m/s
Ingress protection	IP54
Operating Temperature	-20 C to 45 C
Slopes	+/- 30 Degrees
Max step height	300 mm
Lighting	Above 2 lux
Battery capacity	605 Wh
Max battery voltage	58.8V
Typical runtime	90 minutes
Standby runtime	180 minutes
Charger power	400W
Max charge current	7A
Time to charge	120 minutes
Payload max weight	14 kg
Max power per port	150 W
Payload ports	2
Camera type	Projected stereo
Field of view	360 Degrees
Operating range	4 meters
Wi-Fi	802.11
Ethernet	1000Base-T

Table 1: Hardware specification of SPOT, necessary to understand the working environment of SPOT.

2. Cameras
   - Spot exposes five fisheye cameras and five depth cameras as image sources via the on-robot Image service, enabling multi-camera capture with hardware time-sync when multiple sources are requested in a single RPC.
   - Payload Integration (Ricoh Theta); its integration was experimented to get better knowledge about its working.

3.1.2 Software

The software was explored first to find the capabilities of SPOT. The example programs were studied to get better understanding on the communication pattern, the protobuf, gRPC services.

1. OS, Python, SDK version

   - Visual Studio Code was used as the IDE for coding.
   - The SDK Python client supports Ubuntu, Windows, and macOS with Python 3.7–3.10 and includes QuickStart, examples, and full API docs for rapid development. This makes it easy to program robot for specific tasks/applications [7].
   - Open-source python libraries such as curses helped to create the interface. Other basic libraries like math, signal, logging to have diagnostics while testing the program [7].

2. Services

   - Core services to use: Directory and directory-registration, auth, robot-state, robot-command, image, lease, estop, and autonomy services (e.g., GraphNav) are discoverable via the directory service and are listed by default on a running robot [7].
   - Time sync: The Time Sync service coordinates clock skew and ensures command validity windows; the client maintains a background time sync thread after wait_for_sync [7].
   - Some application-based services like the camera service, motor services are used for developing the program.
   - Services play an important role as it saves effort of coding the entire logic from scrap. For e.g. The movement of the robot can be controlled by calling motor service in which it is not always necessary to mention the angle of movement, acceleration and angular acceleration of every motor to have a movement. This calculation is handled by the service.

3.2 Communication

1. SPOT can connect to an available network or host a network for communication with a client system. SPOT behaves as a server system. Both types of communications were tested, and the user can choose any one mode suitable for the site while ensuring good network connection [7,8].
2. gRPC protocol is used for communication which is also open source and resilient against cyber-attacks.

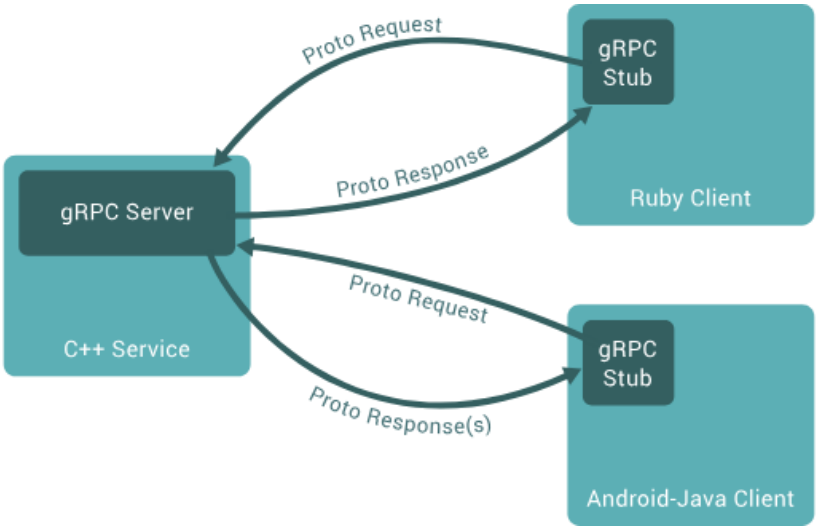

Figure 6: Showing gRPC block diagram to better understand the communication.

3.3 Fiducial tags

1. Fiducials are specially designed images, like QR codes, that Spot uses to match its internal map to the world around it. Fiducials are used to identify docking stations and are required at the beginning of any Auto walk mission [8].
2. Spot recognizes AprilTag fiducials that meet the following requirements:
   - AprilTags in the Tag36h11 set.
   - The default Image size: 146 mm square.
   - Printed on white non-glossy U.S. letter-size sheets (preferably rigid).
   - For 500-series fiducials used with a Spot Dock, the outer boundary of the fiducial sheet should be 200.8 mm wide x 215.5 mm high [8].
3. Though it is recommended by Boston Dynamics to use the standard size, for certain situations other size can be defined in the Admin Console.

3.4 X-box controller

1. Due to time constraints, X-box controller is not integrated in the final program.
2. Nevertheless, the possibility of manually controlling SPOT using X-box controller were explored successfully. (note: only with wired connection to client system) [7]

Figure 7: X-Box controller (with wired and wireless functionality)

4 Implementation

Going through the SPOT documentation and getting inspiration from the example python programs, a new program was created to combine the preset fiducial follow and the WASD program.

4.1 Development Environment and Setup

We implemented the client in Python 3.10 using the SPOT SDK (Python client). Dependencies were managed with venv and installed via pip. The robot connected over WPA2 Wi-Fi as AP, the client uses static IP 160.168.x.x with gRPC endpoints discovered through the Directory service. Further details can be found in the SPOT SDK documentation [7].

4.2 Codebase Organization

The code is divided into three main parts:

1. A python run file to run the program. Saves time instead of manually typing the run command in the terminal. A byproduct for the need to run programs faster during testing stage.
2. The main program with class of Manual control and the interface. Main properties like, lease control, motor control, ESTOP, battery percentage and the key mapping are shown in the interface. The class consisting of the interface also deals with the keyboard inputs and the movement of the robot from the key inputs.

3. The third file consists of the logic for fiducial follow along with the camera display logic. The features in this file are toggled with a key in the main program.
4. This distribution is done to have clear distinction between the various functions of the program.

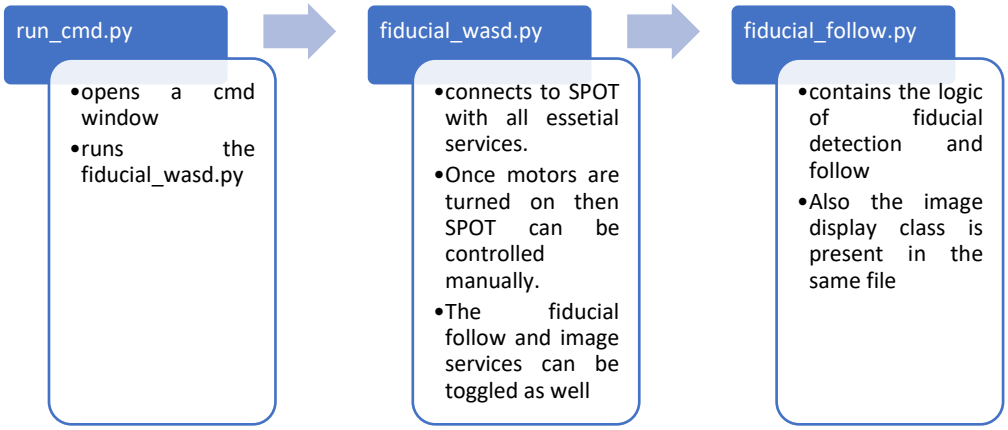

Figure 8: Code Organization. Implemented by considering the Software engineering principles.

4.3 Core Modules and Key Algorithms

4.3.1 Perception: Fiducial Detection

1. Camera source: the fisheyes cameras that stream the video
2. Detection: SPOT has self-detection of fiducial tags. No need of detecting them from the camera feed. Reduces lag.
3. Coordinate handling: Transforming tag pose to robot frame / world frame.
4. This process does not require and additional payload and works effectively with just SPOT.

4.3.2 Navigating to the fiducial.

1. Using services from “bosdyn. client” to limit the speed and acceleration and find the heading from the transformed coordinates to move towards the fiducial. The world frame is preferred as it provides a robust heading calculation and without jittery movements.

4.3.3 Manual control

1. Sinc the fiducial follow is called inside the main manual control. It is possible to have a manual override control even if the SPOT is following a fiducial.

2. The manual input always wins over the fiducial signal almost instantly, and the same applies for resuming fiducial follow within a second if the tag is detected. Hence the importance is always to the operator.

4.3.4 Tasks as per the fiducial tag.

1. Since the world object fiducial detection service can know the code of the fiducial. Based on that code, a task can be assigned to SPOT. This allows SPOT to first detect the code and then do some value-added tasks.

4.4 User Interface and error handling

1. Curses library is used to create a rudimentary but clear user interface to know the functions that can be performed by SPOT. It was selected for being open source and very easy to implement changes. All the error logs can also be seen on the same window to troubleshoot the problems faced by SPOT.
2. All the other Edge cases are taken care with proper coding procedure in both the files.

5 Experiments and observations

In the process to develop the program, more than one way of getting the results were tested and observed. Below is a list of such methods that were considered, worked upon and tested to include into the final code. The conclusions are also derived for each case.

5.1 Detecting fiducials using AprilTag detection

1. To understand the difference between the world object detection and AprilTag detection, the AprilTag library was also tested to detect fiducial tags.
2. The key difference is that fiducial tag is detected in the client system hardware rather than SPOT as it can only be done on API level. This caused intense load on the client system as well as on the communication network. Leading to a lag in the command and response of more than three to five seconds, that is not ideal for our application.

5.2 Fiducial detection range

1. SPOT has a high range in terms of distance to detect a fiducial distance. It has been observed to be around 4 meters.
2. But this is not the same in terms of height. For now, the fiducial should be at level with the SPOT and not higher for fiducial follow to work.

5.3 Descending from stairs

1. The code automatically takes care of ascending stairs but is not capable of descending as Boston Dynamics SPOT can only descend the stairs by walking backwards. This is possible in manual mode.
2. In fiducial detection the heading is prioritized to the front camera. An effort was made to switch the priority from front to back camera. But as observed in the height constraint for fiducial detection. It proved more challenging to implement it.

3. During testing, it was partially successful but if the fiducial tag is not in the view of the SPOT, then it got confused and lost balance on the stairs.
4. More complex algorithm for SPOT to detect the stairs needs to be implemented. As it requires more resources and time. The focus was shifted to distinguish between various fiducial codes and assigning tasks to them.

5.4 X-box controller

1. The appeal of using controller to control SPOT is purely the haptic feel the operator gets by the controller. Joystick provides more accuracy in movement as compared to keyboard.
2. As the main, focus of the present project is to develop a program which can be applied to real life, this topic has been kept open for further research.

5.5 Core I/O

1. Core I/O is a second computing system which takes the load of additional computation of sensors so that the main functionalities of SPOT are unaffected.
2. Although, due to technical difficulties with connecting to Core I/O, we contacted Boston Dynamics for help in troubleshooting. Unfortunately, due to time constraints of the project, we moved ahead without including Core I/O in computation and all heavy computation was given to the client system.

Figure 9: SPOT Core I/O. Currently available with SPOT.

5.6 Ricoh Theta Z1 360° camera

1. Ricoh Theta can be used as payload to have another 360° camera view from the SPOT. Although Core I/O will be required to have it implemented with the current SPOT network and data exchange.
2. It can still be connected to the client system and then analysis can be done on the fetched data. But this method is out of the scope of project and has been kept open for further research.

Figure 10: Ricoh Theta Z1 360 degree camera, which has the possibility of being used as the payload.

6 Results and Discussion

1. The solution to the base problem statement has been achieved. i.e. to have a dual functionality program with manual control being on higher priority. Regardless, the fiducial follow will work in the absence of manual input. This reduces the lead time of switching between the modes during operation.
2. The SPOT can detect fiducials <1s with onboard detection and inform it to the user if falling in the detection range (~4m). A lag of 3-5s is detected when using the AprilTag library.
3. Placement height of fiducial tags is important for detection even with a good horizontal detection range.
4. Can be manoeuvred on stairs in manual mode but can only assent stairs in fiducial follow.
5. Robust handling of logs and errors for troubleshooting.
6. User interface designed to prioritise and organize important vital information for the operator.
7. Smoother communication has been achieved at the end of the project without any jittering motion by SPOT.
8. Delay between video stream is minimal (<1s). Allowing control of SPOT by just viewing the cameras.
9. More effort was put in the earlier stage to understand the working of SPOT and how RPC communication works.
10. Decent number of hours were invested to understand the various services of SPOT as they are the backbone of creating any program with the Python SDK.
11. All the learnings have been documented in the forms of ppts as when the project progressed forward.
12. The tests conducted have proved to show high potential of SPOT being used as a companion with onsite personal in AEC industry.

7 Conclusion and Future Work

1. The combination of having a manual control (kind of like a lease when compared to an actual dog) over its own self function is achieved successfully. Thus achieving the goal of creating a companion.
2. This project was a good exposure in using Python libraries and coming up with innovative solution to use SPOT in a construction site environment.
3. Also, the findings from various experiments have been documented for further reference.
4. The aim of this project was also to lay a foundation for further research to carry on and be even more problem solving based than understanding the functioning of the systems present in SPOT.
5. More future work can be done in image recognition to make SPOT more self-independent, or it can be looked from a managerial perspective to keep on developing programs by adding payloads to minimise the labour-intensive work at the construction sites.

8 References

[1] Navon, R., Sacks, R.: 'Assessing research issues in Automated Project Performance Control (APPC)', *Automation in Construction*, 2007, **16**, (4), pp. 474–484

[2] Halder, S., Afsari, K., Chiou, E., Patrick, R., Hamed, K.A.: 'Construction inspection & monitoring with quadruped robots in future human-robot teaming: A preliminary study', *Journal of Building Engineering*, 2023, **65**, p. 105814

[3] 'Critical Infrastructure: Spot Inspects Hamburg Bridge', https://www.robotsdynamic.com/case-studies/critical-infrastructure-spot-inspects-hamburg-bridge

[4] Afsari, K., Halder, S., Ensafi, M., DeVito, S., Serdakowski, J.: 'Fundamentals and Prospects of Four-Legged Robot Application in Construction Progress Monitoring', 274-263

[5] Halder, S., Afsari, K., Serdakowski, J., DeVito, S.: 'A Methodology for BIM-enabled Automated Reality Capture in Construction Inspection with Quadruped Robots'

[6] 'Github repository', https://github.com/abhishekdilippatil/spot-sdk

[7] 'Spot SDK', https://dev.bostondynamics.com/readme

[8] 'Boston Dynamics SPOT', https://support.bostondynamics.com/s/topic/0TOUS0000003syL4AQ/spot

9 Appendix

9.1 How to Operate SPOT

9.1.1 Initiation Checklist

- Clear the area & brief people nearby.
- Ensure an operator has a reachable hardware or software E-Stop.
- Power and battery check.
- Put the tablet or your control laptop on the same network as the robot.
-

9.1.2 Programming Operations

1. Install the Python SDK

1.1. Install Python 3.7–3.10.

1.2. Verify your python install is the correct version. Open a command prompt or start your python IDE:
Then enter

```
$ python --version
Python 3.10.12
```

1.3. **Windows users:** There are two common methods for starting the Python interpreter on the command line. First, launch a terminal: Start > Command Prompt. At the command prompt, enter either:

```
> python.exe
```

Or

```
> py.exe
```

2. Pip Installation

Pip is the package installer for Python. The Spot SDK and the third-party packages used by many of its programming examples use pip to install.

Check if pip is installed by requesting its version:

```
$ python3 -m pip --version
pip 19.2.1 from <PATH_ON_YOUR_COMPUTER>
```

For Windows users:

```
> py.exe -3 -m pip –version
```

3. Install Spot Python packages

With python and pip properly installed and configured, the Python packages are easily installed or upgraded from PyPI with the following command:

```
$ python3 -m pip install --upgrade bosdyn-client bosdyn-mission bosdyn-choreography-client bosdyn-orbit
```

Installing the `bosdyn-client`, `bosdyn-choreography-client` and `bosdyn-mission` packages will also install `bosdyn-api` and `bosdyn-core` packages with the same version. The command above installs the latest version of the packages.

4. Verify your Spot packages installation
Enter this command to verify all the packages

```
$ python3 -m pip list --format=columns | grep bosdyn
```

The Output Should be:

```
bosdyn-api                    5.0.1
bosdyn-choreography-client    5.0.1
bosdyn-choreography-protos    5.0.1
bosdyn-client                 5.0.1
bosdyn-core                   5.0.1
bosdyn-mission                5.0.1
bosdyn-orbit                  5.0.1
```

For Windows Users:

```
> python3 -m pip list --format=columns | findstr bosdyn
```

For more accurate step and troubleshooting you can refer to:
https://dev.bostondynamics.com/docs/python/quickstart